

1 AES avec openssl

L'objectif de cette partie est de chiffrer une image en ligne de commande (pas en Python, donc) à l'aide des outils fournis par la librairie `openssl`.

L'en-tête d'une image au format `ppm` possède trois lignes :

- la première contient les caractères P6 (pour indiquer le format de fichier) ;
- la seconde contient les dimensions en pixels ;
- la troisième la valeur maximale d'un pixel (255 dans notre cas).

La suite contient les pixels, ligne par ligne, codés chacun sur trois octets (pour les trois couleurs).

Pour plus d'informations, sur les fonctions utilisées, utiliser le manuel (`manpage`).

Exercice 1. On considère l'image `tux.ppm`.

1. Avec la commande `head`, mettre les 3 premières lignes de votre fichier ppm dans un fichier `header.txt`.
2. Avec la commande `tail` mettez tout le fichier ppm sauf les 3 premières lignes dans un fichier `body.bin`.
3. Chiffrer le fichier `body.bin` avec les différents algorithmes `aes-128-cbc`, `aes-128-ecb`, `aes-128-ctr` et nommer les résultats du type `tux-aes-128-cbc.enc`, etc.
4. Remettre l'en-tête `header.txt` au début des fichiers obtenus à la question précédente pour avoir des images chiffrées, et observer le résultat.

2 Cryptographie symétrique — Chiffrement par blocs

Dans la suite, nous allons utiliser `AES-128` avec différents modes opératoires.

Le chiffrement par bloc utilise différents modes opératoires (dont ECB, CBC, OFB, CTR). On souhaite implanter certains de ces modes en Python. On va utiliser la bibliothèque `pycryptodome` qui fournit de nombreuses primitives cryptographiques.

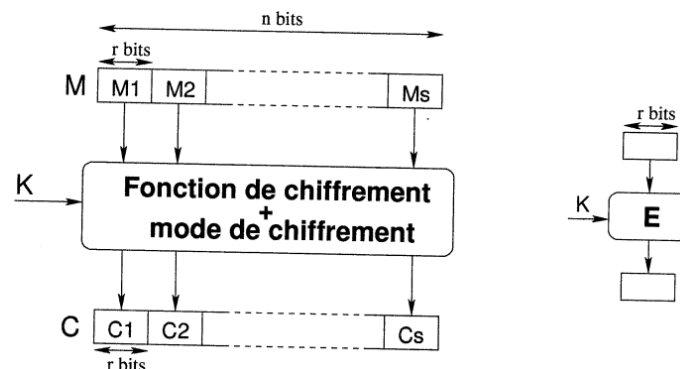


FIGURE 1 – Chiffrement par bloc

Au besoin exécuter :

```
1 ~$ pip3 install pycryptodome
```

La bibliothèque fournit beaucoup plus de fonctions que celles qu'on utilise dans ce TP. En particulier, pour la plupart des questions qui suivent, il existe une fonction dans la bibliothèque qui effectue la tâche demandée. **L'objectif n'est pas d'utiliser ces fonctions, mais de les coder soi-même.**

2.1 Type de données

On souhaite manipuler des données sous forme de bits, ou plus précisément d'octets. Pour cela, on utilise les *bytestrings* Python.

Une bytestring s'écrit comme une chaîne de caractère, préfixée du caractère « b » : par exemple

```
1 m = b'ceci est une bytestring'
```

Les caractères de la chaîne sont

- soit des caractères **ASCII** ;
- soit directement un octet quelconque spécifié par ses huit bits. Pour cela, on écrit `\xhh` où `hh` est l'écriture hexadécimale (base 16) de l'entier représenté par les huit bits.

Par exemple, l'octet `00101101` s'écrit `2d` en hexadécimal. En effet, `1101` vaut 134 et que `a`, `b`, ..., `f` représente 10, 11, ... 15.

Pour entrer cet octet, on écrit donc `b'\x2d'`.

Remarque. De manière assez surprenante, si `s` est une bytestring, la syntaxe `s[i]` renvoie le caractère d'indice `i` sous la forme d'un entier.

Ainsi, si `s = b'\x2d'`, `s[0]` renvoie 45.

À l'inverse, `s[a:b]` renvoie bien une *bytestring*.

2.2 Padding

Pour utiliser un mode opératoire, la chaîne en entrée doit posséder un nombre de bits qui soit un multiple de 128. Il faut donc *padder* la chaîne en entrée.

On ajoute les bits `100...0` de sorte que la chaîne ait une longueur multiple de 128 (en bits).

Attention, on ajoute au moins un `1`, donc si la longueur de la chaîne est au départ un multiple de 128, on l'allonge de 128 bits exactement.

Exercice 2.

1. Écrire une fonction `pad(s)` qui prend en entrée une *bytestring* `s` et renvoie une *bytestring* paddée.
2. Écrire une fonction `unpad(s)` qui retire le padding de `s`.
Si `s` n'a pas été paddée correctement, la fonction soulève une exception, grâce à l'instruction `raise ValueError("message non correctement paddé")`.
3. Tester en ajoutant du *padding* puis en le retirant, à des chaînes de diverses longueurs.

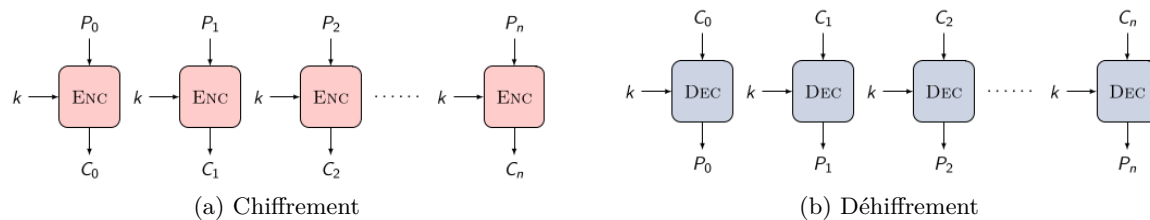


FIGURE 2 – Mode ECB

Pour implanter les modes opératoires, on fournit les fonctions `Enc` et `Dec` qui prennent en entrée un bloc de 128 bits (16 octets) et une clef de la même taille, et renvoient ce bloc chiffré ou déchiffré grâce à `AES`.

```

1  from Crypto.Cipher import AES
2  from Crypto.Random import get_random_bytes
3  from Crypto.Random.random import randrange
4  from Crypto.Util.strxor import strxor as xor
5  from Crypto.Util.number import bytes_to_long, long_to_bytes
6  from hashlib import md5, sha3_256, sha1
7
8  def Enc(bloc, clef):
9      if not isinstance(bloc, bytes) or not isinstance(clef, bytes):
10         raise TypeError("le bloc et la clef doivent être de type bytes")
11      if len(bloc) != 16 or len(clef) != 16:
12         raise ValueError("le bloc et la clef doivent faire 16 octets")
13      return AES.new(clef, AES.MODE_ECB).encrypt(bloc)
14
15  def Dec(bloc, clef):
16      if not isinstance(bloc, bytes) or not isinstance(clef, bytes):
17         raise TypeError("le bloc et la clef doivent être de type bytes")
18      if len(bloc) != 16 or len(clef) != 16:
19         raise ValueError("le bloc et la clef doivent faire 16 octets")
20      return AES.new(clef, AES.MODE_ECB).decrypt(bloc)

```

2.3 Mode ECB

Dans le mode `ECB`, chaque bloc est chiffré indépendamment des autres. Voir figure 2 page 3.

Exercice 3.

1. Implanter le chiffrement dans le mode opératoire `ECB`, sous la forme d'une fonction `Enc_ECB(clair, clef)` qui renvoie le chiffré de clair avec la clef, en utilisant `AES` en mode `ECB`.
2. Implanter de même le déchiffrement, sous la forme d'une fonction `Dec_ECB(chiffre, clef)` qui renvoie le clair de chiffre avec la clef, en utilisant `AES` avec le mode `ECB`.
3. Tester vos fonctions en chiffrant et déchiffrant des messages!

2.4 Mode CBC

Exercice 4. Le fonctionnement du mode `CBC` est rappelé sur la figure 3 page 4.

Reprendre les questions de l'exercice précédent pour écrire les fonctions `Enc_CBC` et `Dec_CBC`.

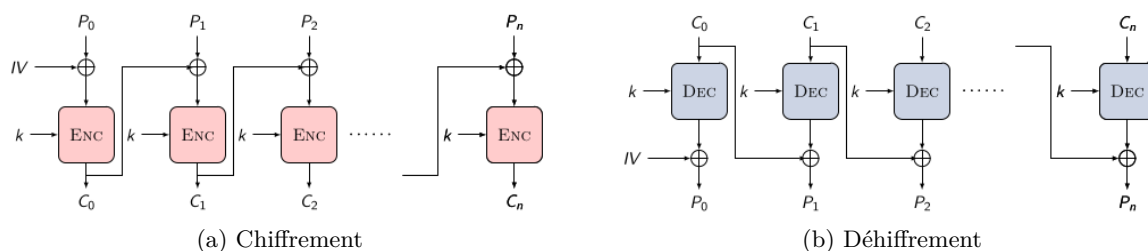


FIGURE 3 – Mode CBC

Pour effectuer un `XOR` bit-à-bit de deux bytestrings, utiliser la fonction fournie.

Pour créer le vecteur d'initialisation `IV`, utiliser la fonction `get_random_bytes(n)` qui permet de générer une bytestring aléatoire de n octets.

3 Attaque sur ECB

On cherche maintenant à trouver quelle image mystère se cache dans `image.ecb.ppm`. La difficulté est qu'on ne connaît pas les dimensions de l'image et donc l'en-tête à rajouter.

Rappels. L'en-tête d'une image au format PPM possède trois lignes : la première contient les caractères P6 (pour indiquer le format de fichier), la seconde contient les dimensions en pixels, et la troisième la valeur maximale d'un pixel (255 dans notre cas). La suite contient les pixels, ligne par ligne, codés chacun sur trois octets (pour les trois couleurs).

Exercice 5.

- (a) Si on dispose d'une image de dimension $L \times H$ en pixels, quelle est la taille du fichier en octet, en ignorant l'en-tête ?
- (b) Dans l'autre sens, si on dispose d'un fichier de N octets, de combien de pixels est constituée l'image (environ, en estimant vaguement la taille de l'en-tête) ?
- (c) Si on sait qu'une image fait N octets, quelles peuvent être ses dimensions ?
- À l'aide de la question précédente, et d'un peu de tâtonnement, trouver quelle image se cache derrière le fichier `image.ecb.ppm`.

Aide : Calculer le nombre approximatif de pixels de l'image, estimer alors assez finement la taille en octet de l'en-tête, puis raffiner le calcul du nombre de pixels (qui est toujours une approximation à cause du padding). Produire beaucoup de fichiers de différentes dimensions et les parcourir visuellement.